TIA — Technical Framework Document

Enterprise Change Impact Intelligence Platform

Version: 1.0
Status: Proposed technical baseline
Companion: PRD.md v2.0 · ARCHITECTURE.md · UI-PRD.md
Primary domain: Banking and financial-services engineering
Architecture style: Modular, evidence-driven, API-first platform
Last updated: 2026-10-28

---

1. Purpose

The Technical Framework Document defines the engineering framework for implementing TIA. It establishes the technology baseline, system boundaries, internal modules, graph model, processing contracts, APIs, evidence standards, security controls, deployment approach, and implementation sequence.

TIA must answer:

> Given a proposed code, API, event, or data-schema change, which downstream components may be affected, what evidence supports each finding, what verification is required, and what remains unknown?

The system must provide explainable, reproducible technical impact analysis rather than simply displaying dependencies or generating an AI-written summary.

2. Architectural principles

1. **Evidence before conclusions:** Every material impact finding must link to supporting evidence.
2. **Deterministic analysis:** Dependency traversal, revision comparison, and policy evaluation must be reproducible for the same inputs and configuration.
3. **Explicit uncertainty:** Missing, stale, inferred, and incomplete evidence must remain distinguishable.
4. **Version-aware relationships:** Dependencies must be associated with relevant source revisions, contract versions, and observation windows.
5. **Incremental processing:** A new change should update affected evidence instead of requiring a full rebuild unnecessarily.
6. **Separation of concerns:** Source extraction, identity resolution, graph persistence, impact analysis, and policy evaluation are separate responsibilities.

7. **No false completeness:** A successful scan of one repository does not establish complete knowledge of the enterprise.
8. **Enterprise security:** Access to source code, evidence, analysis reports, and administrative operations must be authorized and audited.

3. Reference technology stack

The following is the proposed baseline, not a claim that these technologies have already been implemented or benchmarked.

Layer	Proposed technology	Responsibility
User interface	React, TypeScript	Impact reports, evidence exploration, service views
API layer	Python, FastAPI, Pydantic	API contracts, validation, authorization integration
Source analysis	Python AST and Tree-sitter	Symbols, functions, imports, calls, framework-specific extraction
Contract analysis	OpenAPI parsers and schema-diff utilities	API and schema change detection
Metadata database	PostgreSQL	Repositories, jobs, revisions, policies, analysis snapshots
Dependency graph	RDF graph store or property graph	Typed dependencies and evidence-backed traversal
Evidence storage	S3-compatible object storage or controlled filesystem	Source references, manifests, reports and scan artifacts
Background execution	PostgreSQL-backed job queue initially	Asynchronous scans and analysis jobs
Source integration	GitHub App and webhooks	PR events, commits, repository access
Testing	pytest and API/integration tests	Extraction, graph, impact and policy verification
Delivery	Containerized services and CI/CD	Repeatable build, deployment and release

Technology decision rule

Do not introduce Kafka, Kubernetes, a vector database, multiple graph databases, or an LLM orchestration framework by default. Add infrastructure only when measured scale, availability, or operational requirements justify it.

The graph-store choice must be validated against representative queries, update frequency, traversal depth, evidence requirements, and deployment constraints before committing to a production implementation.

4. Logical system architecture

The platform is organized into six logical modules.

4.1 Connector module

Responsibilities:

- Register repositories and source-control integrations.
- Retrieve PR metadata, commit SHAs, diffs, and permitted source content.
- Retrieve contract specifications, CI results, test reports, deployment metadata, and runtime observations when connected.
- Record connector permissions, collection timestamps, failures, and source coverage.

Each connector must expose consistent status and error information.

4.2 Extraction module

Responsibilities:

- Parse source files and identify symbols, functions, classes, endpoints, imports, and supported call relationships.
- Identify API and event contracts.
- Extract test definitions and test-to-code evidence where available.
- Extract migration and data-access relationships when supported by the language or framework adapter.
- Preserve source locations and revision identifiers.

Extraction must be language-adapter based. The initial implementation should support Python and selected API frameworks, with additional languages introduced through explicit adapters.

4.3 Identity resolution and graph writer

Responsibilities:

- Resolve entities across repositories and revisions.
- Associate endpoints with service identities and contract definitions.
- Normalize identifiers and relationship types.
- Store provenance, confidence status, and temporal validity.
- Invalidate or supersede stale relationships when source evidence changes.
- Prevent unverified candidate relationships from silently becoming authoritative.

The identity-resolution module must distinguish an entity's identity from the evidence supporting a relationship to another entity.

4.4 Impact-analysis engine

Responsibilities:

1. Resolve the base and proposed revisions.
2. Identify changed entities.
3. Find directly related components.
4. Traverse relevant dependency relationships within configured bounds.
5. Correlate affected components with tests, contracts, deployments, and runtime evidence.
6. Classify findings by impact type, severity, evidence status, and limitations.
7. Record unresolved dependencies and analysis coverage.
8. Produce an immutable analysis snapshot.

The engine must not interpret a missing graph edge as proof that a dependency does not exist.

4.5 Policy engine

Responsibilities:

- Evaluate configured release and verification policies.
- Determine whether mandatory evidence or tests are missing.
- Apply policy scope, conditions, severity, and outcome.
- Return explicit results: `PASS`, `FAIL`, `NOT_EVALUATED`, or `REQUIRES_REVIEW`.
- Record the policy version and evidence used for each decision.

A policy evaluation must not convert an incomplete analysis into a pass.

4.6 Reporting and API module

Responsibilities:

- Provide analysis summaries and detailed findings.
- Serve impacted-component lists and evidence paths.
- Expose test, contract, runtime, and deployment evidence.
- Return unknowns, limitations, and coverage information.
- Support UI rendering, PR reporting, and export.

These modules may initially run in one deployable backend with clear internal interfaces. Separate services should be introduced only when independent scaling or isolation is required.

5. Canonical technical data model

5.1 Core entities

Entity	Purpose
Repository	Source-control repository
Commit	Immutable source revision
File	Source file at a revision
FunctionVersion	Function or method at a particular revision
APIEndpoint	Exposed API operation
Event	Published or consumed event
Schema	Versioned API, event, or data schema
DataEntity	Logical business or technical data entity
Table	Physical database table
TestCase	Defined verification test
TestRun	Execution of a test in a specific build or run
Service	Deployable or logical application service
Deployment	Release of an artifact into an environment
Incident	Operational incident record
Owner	Responsible team or organizational identity
Policy	Versioned verification or release rule
EvidenceRecord	Source supporting an entity or relationship
AnalysisRun	Immutable impact-analysis execution

5.2 Core relationship types

Relationship	Meaning
`CONTAINS`	Repository or file contains a technical entity
`CALLS`	One function invokes another
`EXPOSES`	Service exposes an API endpoint
`PRODUCES`	Component publishes an event
`CONSUMES`	Component consumes an event
`USES_SCHEMA`	Component uses a schema
`READS`	Function reads a data entity or table
`WRITES`	Function writes a data entity or table
`TESTS`	Test verifies a component or behavior

Relationship	Meaning
`COVERS`	Coverage evidence associates execution with code
`BUILT_FROM`	Artifact was built from a source revision
`DEPLOYED_AS`	Service or component is deployed as an artifact
`OBSERVED_CALL_TO`	Runtime evidence records a call relationship
`OWNED_BY`	Component has an assigned owner
`ASSOCIATED_WITH`	Incident or other evidence is associated with a component

These relationships must have explicit semantics. For example, `CALLS` extracted from source code is not equivalent to `OBSERVED_CALL_TO` recorded from production telemetry.

5.3 Evidence and provenance model

Every important relationship should carry, directly or through a linked evidence record:

- Evidence identifier and evidence type.
- Source repository, file, line range, or external record.
- Observed revision or artifact version.
- Collection and analysis timestamps.
- Validity interval where applicable.
- Extraction method and tool version.
- Evidence status and confidence, where meaningful.
- Freshness and known limitations.

Use explicit statuses such as:

- `SOURCE_CONFIRMED`
- `CONTRACT_CONFIRMED`
- `RUNTIME_OBSERVED`
- `CORROBORATED`
- `INFERRED`
- `UNKNOWN`
- `STALE`

These statuses describe the nature of the evidence, not an absolute guarantee of correctness.

6. Change-analysis processing contract

The primary workflow is a proposed code change submitted through a pull request.

1. **Accept the request:** Record the repository, PR, base SHA, proposed SHA, and analysis configuration.
2. **Validate inputs:** Confirm revision availability, repository permissions, and required integration status.

3. **Extract the diff:** Identify changed files, symbols, endpoints, schemas, and other supported entities.
4. **Update evidence:** Reuse valid unchanged evidence and refresh affected evidence.
5. **Resolve identities:** Connect changed entities to known components and dependency relationships.
6. **Traverse dependencies:** Follow supported relationships using configured depth and scope limits.
7. **Correlate verification:** Identify associated tests, coverage, contract checks, runtime observations, and deployments.
8. **Classify findings:** Determine the affected component, impact category, evidence status, severity, and limitations.
9. **Evaluate policies:** Apply the configured rules to the available evidence and analysis coverage.
10. **Persist the result:** Save the analysis inputs, outputs, evidence references, policy version, and limitations.
11. **Publish the report:** Return results to the API, UI, and optionally the PR check.

An analysis must identify the exact revisions evaluated. If the proposed revision changes while analysis is running, the result must remain associated with the original revision pair and must not be presented as an assessment of the newer change.

7. Impact-analysis result contract

Each analysis response should include:

Field	Description
`analysis_id`	Unique analysis identifier
`base_revision`	Exact starting revision
`proposed_revision`	Exact proposed revision
`status`	Complete, partial, failed, or running
`coverage_summary`	Sources and evidence successfully evaluated
`findings`	Impacted components and supporting evidence
`unknowns`	Unresolved dependencies and missing evidence
`policy_results`	Individual policy outcomes
`generated_at`	Analysis creation timestamp
`configuration_version`	Rules and configuration used

Each finding should include a component identifier, impact type, severity, evidence status, relationship path, source references, associated test evidence, limitations, and recommended verification checks.

Recommendations must be traceable to evidence and rules. They must not be presented as proof that a test is mandatory unless the applicable policy establishes that requirement.

8. API framework

Proposed REST endpoints:

Endpoint	Purpose
`POST /api/v1/repositories`	Register a repository
`POST /api/v1/scans`	Submit a source or evidence scan
`GET /api/v1/scans/{scan_id}`	Retrieve scan status and coverage
`POST /api/v1/impact-analyses`	Create an impact analysis
`GET /api/v1/impact-analyses/{analysis_id}`	Retrieve the report
`GET /api/v1/impact-analyses/{analysis_id}/paths`	Retrieve evidence-backed impact paths
`GET /api/v1/components/{component_id}/dependencies`	Retrieve component dependencies
`GET /api/v1/policies`	List available policies
`POST /api/v1/policy-evaluations`	Evaluate policies against an analysis

API conventions:

- Validate request and response models.
- Use stable error codes and structured error responses.
- Make job submission idempotent where appropriate.
- Return job identifiers for asynchronous processing.
- Apply pagination and traversal limits.
- Enforce authorization at the API boundary and on underlying resources.
- Version externally consumed contracts.

9. Persistence and consistency

Use PostgreSQL for transactional metadata, job state, policy definitions, and analysis records. Use a graph store for typed relationships and bounded dependency traversal. Store large evidence artifacts outside the graph and reference them through stable identifiers.

The implementation must define:

- A canonical identifier strategy.
- Transaction boundaries for graph and metadata updates.
- Retry and duplicate-processing behavior.
- Idempotent ingestion operations.
- Rules for superseding or invalidating stale evidence.

- Recovery behavior when one persistence layer succeeds and another fails.
- Retention and deletion rules for source-derived evidence.

Graph updates and relational updates must not be assumed to be atomic across separate systems. Use explicit processing states, retryable operations, and reconciliation checks.

10. UI integration framework

The UI consumes the API rather than independently reconstructing impact logic.

The principal screens are:

1. Overview.
2. Change Impact Report.
3. Impact Path Explorer.
4. Services.
5. Contracts.
6. Tests and Evidence.
7. Deployments and Incidents.
8. Policies.
9. Administration.

The report must expose the finding, supporting evidence, dependency path, verification status, and unresolved questions together.

The UI must differentiate:

- No findings from failed analysis.
- Missing evidence from a failed test.
- A test's existence from a test's execution and pass status.
- Source-inferred dependencies from runtime-observed dependencies.
- A policy pass from a policy that was not evaluated.

Graph visualization is a supporting interaction, not the sole representation of analysis results. Users must also be able to inspect findings in accessible tables and evidence panels.

11. Security and governance

The enterprise implementation must support:

- Least-privilege repository and integration permissions.
- Role-based access to analysis, administration, and policy management.
- Secret storage through an approved secrets manager.
- Encryption in transit and at rest.
- Audit records for ingestion, analysis, exports, and administrative changes.
- Configurable retention and deletion of source-derived evidence.
- Tenant or organization isolation where required.

- Controlled outbound network access.
- Protection against malicious repository content and unsafe parser execution.
- Explicit handling of credentials, secrets, and sensitive data discovered in source files.

The deployment model should support a private environment for banks that cannot permit source code or engineering metadata to leave their controlled boundary.

12. Reliability and performance

The system must be designed and tested for:

- Large repositories and multi-repository dependency traversal.
- Partial connector failures.
- Parser failures and unsupported language constructs.
- Duplicate or delayed webhook events.
- Stale evidence and changing source revisions.
- Interrupted background jobs.
- Graph traversal limits and incomplete result sets.
- Concurrent analysis requests.
- Retry and recovery after persistence failures.

Performance targets should be set using a representative bank application corpus. Initial targets should measure scan duration, impact-report latency, memory use, graph-query cost, and concurrent analysis capacity.

Do not advertise enterprise-scale performance until those measurements have been collected.

13. Testing framework

Unit tests

Validate parsers, identifier normalization, relationship extraction, evidence status classification, and policy rules.

Integration tests

Validate connectors, database persistence, graph writes, graph traversal, and API responses.

Golden-corpus tests

Maintain representative source repositories and known dependency relationships. Compare expected and actual impact findings across releases.

Regression tests

Verify that parser or rule changes do not silently remove previously established dependencies or change policy outcomes.

Security tests

Test authorization boundaries, secret handling, malicious input, dependency vulnerabilities, and audit behavior.

End-to-end tests

Submit a representative PR, run the analysis, inspect the evidence paths, and verify the resulting UI report and policy outcomes.

Evaluate recall, precision, evidence completeness, false-positive rate, and analysis coverage separately. A single accuracy score is insufficient.

14. Observability and operations

Record structured logs, metrics, traces, and job events for:

- Connector success and failure.
- Extraction duration and parser errors.
- Entities and relationships created or superseded.
- Identity-resolution failures.
- Graph traversal size and duration.
- Analysis completeness and unknown counts.
- Policy outcomes.
- Queue depth, retries, and processing failures.
- Evidence freshness and source coverage.

Operational dashboards must distinguish platform health from analysis completeness. A healthy service can still produce an incomplete report because an external source is unavailable.

15. Implementation phases

Phase 1 — Technical foundation

- Repository registration and GitHub integration.
- Base/head revision handling.
- Python source extraction.
- Initial canonical entity and relationship model.
- Metadata persistence and graph-store evaluation.
- Evidence provenance and scan status.

Phase 2 — Core impact analysis

- Changed-symbol resolution.
- Bounded graph traversal.
- Cross-repository relationships.
- Evidence-backed impact findings.
- Unknowns and coverage reporting.
- API and CLI report output.

Phase 3 — Verification intelligence

- OpenAPI and schema comparison.
- Test association and execution evidence.
- Initial policy engine.
- UI impact report and path explorer.
- GitHub PR reporting.

Phase 4 — Operational intelligence

- Deployment and runtime integrations.
- Incident correlation.
- Service ownership and criticality.
- Historical analysis comparison.
- Enterprise audit and operational controls.

Phase 5 — Production hardening

- Load and resilience testing.
- Security review.
- Recovery and reconciliation procedures.
- Retention enforcement.
- Pilot evaluation against agreed acceptance criteria.

Phases are a proposed delivery sequence; their completion is not implied by this document.

16. Acceptance criteria

The technical framework is considered implemented when:

1. An analysis is tied to exact base and proposed revisions.
2. Changed source entities can be resolved to supported graph entities.
3. Impact traversal returns typed, inspectable relationship paths.
4. Material findings expose their supporting evidence.
5. Evidence provenance and validity are preserved.
6. Missing, stale, and inferred relationships are explicitly represented.
7. Partial and failed analyses cannot masquerade as complete zero-impact reports.
8. Test existence, execution, result, and coverage remain distinct.
9. Policy evaluations record the policy version and outcome.
10. Results are available through documented APIs and the UI.

11. Ingestion and analysis operations support retries and duplicate handling.
12. Security, regression, and performance tests cover the agreed pilot scope.

17. Technical decisions still requiring validation

Before the production baseline is frozen, the engineering team must decide:

- RDF versus property graph based on representative workload tests.
- Supported languages and frameworks for the initial release.
- Canonical identity rules for APIs, functions, schemas, and services.
- The evidence confidence and freshness model.
- Required depth and scale limits for impact traversal.
- The initial release-policy language and execution semantics.
- Runtime and deployment integration scope.
- Data retention, tenant isolation, and private deployment requirements.
- Pilot corpus, baseline measurements, and acceptance thresholds.

18. Definition of success

TIA succeeds when engineering teams can answer, with inspectable evidence:

- What changed?
- Which components may be affected?
- Why does TIA believe they are affected?
- Which tests and contract checks provide verification?
- Which production observations are relevant?
- What evidence is missing or stale?
- Which release policies passed, failed, or require review?

Core engineering principle: Every important impact finding must explain what is affected, why the relationship is credible, where the evidence comes from, and what remains unknown.